

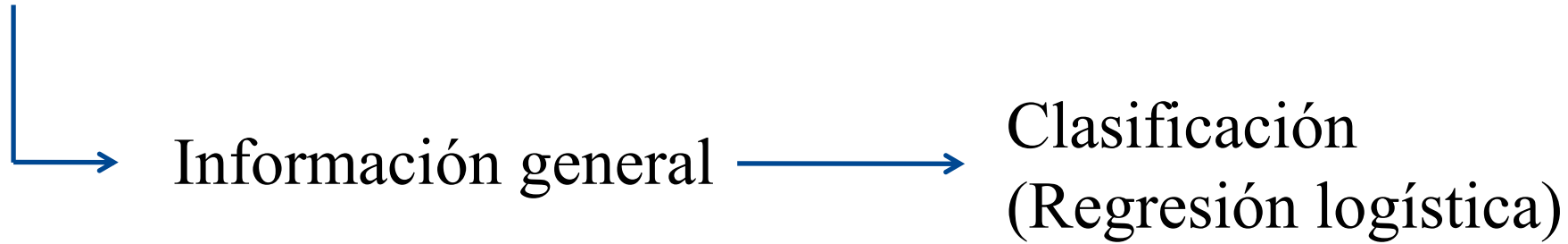
TC3002B – M2. IA

Clasificación

Iván Amaya



Presentar

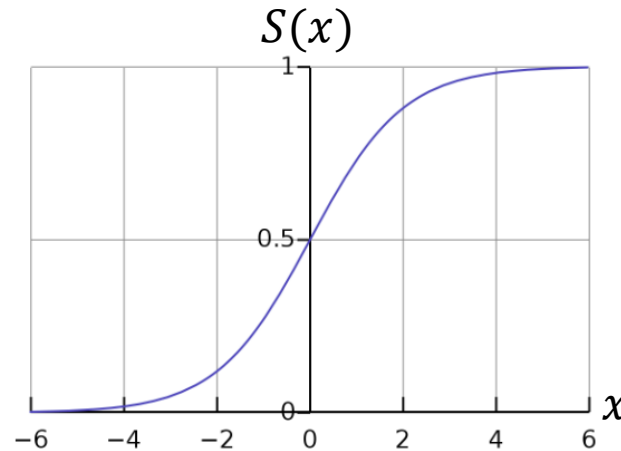


❖ CLASIFICACIÓN

- ✦ IDEA GENERAL
- ✦ CROSS-ENTROPY (A.K.A. LOG-LOSS)
- ✦ GRADIENTE DESCENDENTE
- ✦ MATRIZ DE CONFUSIÓN
- ✦ CLASIFICACIÓN MULTICLASE Y MULTIVARIABLE
- ✦ IMPLEMENTACIÓN CON SCIKIT-LEARN

- † La idea con un algoritmo de clasificación binario es poder identificar si una **muestra** en particular (un **dato**) **pertenece** (1) o **no** (0) a una clase (categoría).
- † Esto quiere decir que la variable dependiente de cada uno de los datos que usaremos para entrenar nuestros modelos es simplemente un **uno** o un **cero**.
- † En principio, podríamos usar una regresión lineal para aprender de los datos, pero hay varios problemas:
 - ‡ Nuestras predicciones **NO** corresponden a valores discretos (no están acotadas)
 - ‡ Mientras más nos alejamos de los datos, más extremo será el comportamiento
 - ‡ La calidad del modelo cambia rápidamente según la separación de los datos
- † Por ello, lo que se hace es **transformar** el modelo lineal a través de la función **sigmoide**.

- † La función **sigmoide** es una función que se encuentra acotada entre cero y uno:



$$S(x) = \frac{1}{1 + e^{-x}}$$

- † Esto quiere decir que **NO** importa si los valores son muy **grandes** o muy **pequeños**, el valor transformado siempre estará entre **cero** y **uno**.
- † Además, este valor puede tomarse como una probabilidad asociada con el hecho de **pertenecer a la clase** (i.e., que la salida sea **1**). Por ejemplo:
- ‡ La probabilidad de que un dato pertenezca a la clase es virtualmente del **100%** cuando el dato (x) es **6**, pero del **0%** cuando $x = -6$.

- † Nuestra función de hipótesis para clasificación binaria es, entonces, la **transformación** de nuestra regresión lineal. Es decir:

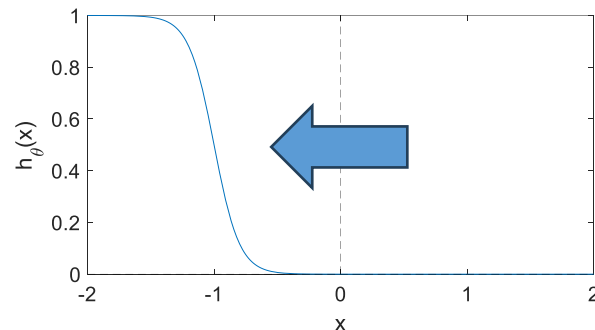
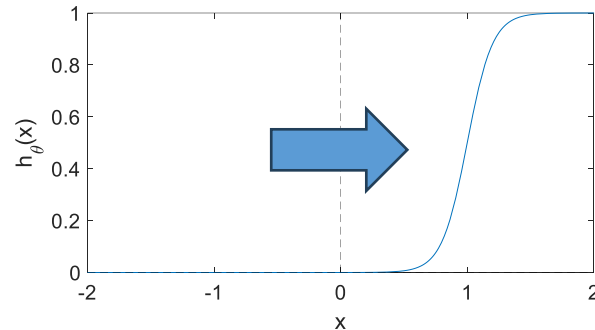
$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$

- † Y, la función de costo corresponde con la función conocida como *cross-entropy* o *log-loss*:

$$J_{\theta} = J(\theta_0, \theta_1) = -\frac{1}{n} \sum_{i=1}^n (y_i \ln(h_{\theta}(x_i)) + (1 - y_i) \ln(1 - h_{\theta}(x_i)))$$

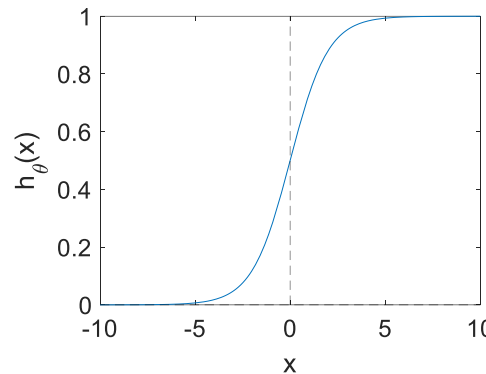
- ‡ Note que los términos que están dentro de la sumatoria se anulan dependiendo de si la clase ‘real’ (y_i) es 0 o 1.
- † Podemos usar esta función de costo para monitorear el comportamiento del modelo a medida que lo entrenamos.

- † Así como con la regresión lineal, los parámetros iniciales definirán la forma general del comportamiento del clasificador:

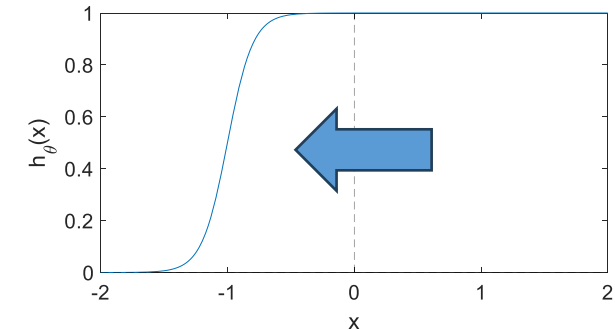


$$\theta_0 = -10$$

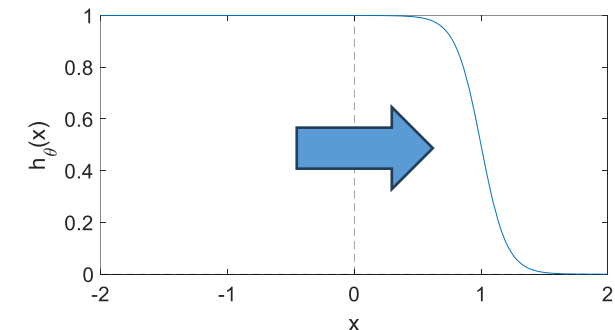
$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$



$$\theta_0 = 0 \quad \theta_1 = 1$$



$$\theta_1 = 10$$



$$\theta_1 = -10$$

$$\theta_0 = 10$$

- † Aunque la forma puede cambiar con las iteraciones, puede llegar a tomar un buen número de ellas (dependiendo del valor del *learning rate*).

† Recordemos que el método de GD consiste en iterar usando:

$$\theta_j = \theta_j - \alpha \frac{\partial J_{\theta}}{\partial \theta_j} \quad \forall j \in \{0, 1, \dots\}$$

† Por tanto, podemos tomar la función de costo y hacer las derivadas parciales para llegar a la expresión final:

$$\theta_0 = \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_i) - y_i) \quad \theta_k = \theta_k - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_{i,k}) - y_i) x_{i,k} \quad \forall k \in \{1, 2, \dots\}$$

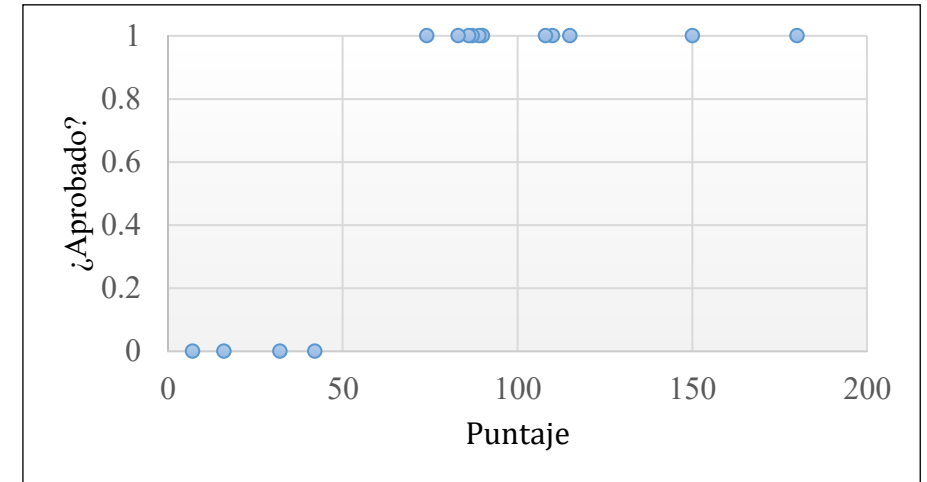
† No nos detendremos en la demostración por cuestiones de tiempo, pero pueden intentar hacer las derivadas por su cuenta y validar las ecuaciones.

† Note que la estructura es idéntica al caso de regresión lineal, pero ahora debemos usar la función de hipótesis de la regresión logística:

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots)}}$$

† Calcule la función de costo para $\theta = [-5, 0.1]$:

Puntaje	¿Aprobado?	$h_{\theta}(x_i)$	$\ln(h_{\theta}(x_i))$	$1 - h_{\theta}(x_i)$	$\ln(1 - h_{\theta}(x_i))$
90	1				
89	1				
16	0				
87	1				
86	1				
42	0				
74	1				
83	1				
32	0				
7	0				
110	1				
108	1				
115	1				
180	1				
150	1				

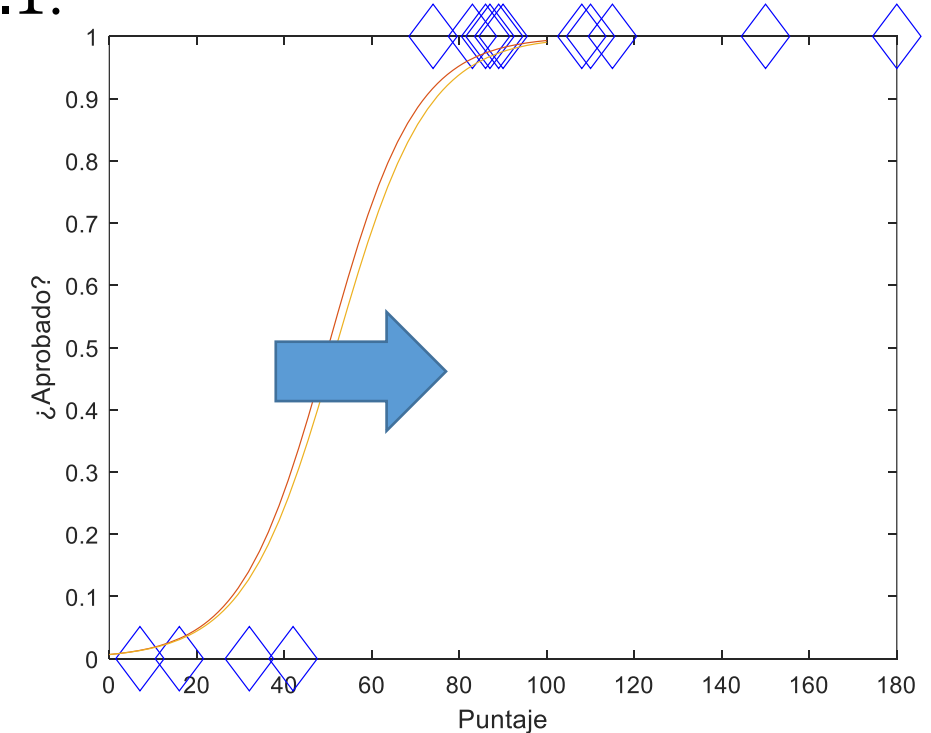


$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots)}}$$

$$J_{\theta} = -\frac{1}{n} \sum_{i=1}^n (y_i \ln(h_{\theta}(x_i)) + (1 - y_i) \ln(1 - h_{\theta}(x_i)))$$

† Si $\theta = [-5, 0.1]$, itere una vez con GD y $\alpha = 0.1$:

Puntaje	¿Aprobado?	$h_{\theta}(x_i)$	$h_{\theta}(x_i) - y_i$	$\Delta \cdot x_i$
90	1			
89	1			
16	0			
87	1			
86	1			
42	0			
74	1			
83	1			
32	0			
7	0			
110	1			
108	1			
115	1			
180	1			
150	1			



$$\theta_0 = \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_i) - y_i)$$

$$\theta_k = \theta_k - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_{i,k}) - y_i) x_{i,k} \quad \forall k \in \{1, 2, \dots\}$$

† ¿Cómo le va a un alumno que obtiene 70?

‡ Para determinarlo debemos usar el modelo:

$$h_{\theta}(x) = \frac{1}{1+e^{-(\theta_0+\theta_1x)}} \rightarrow \begin{matrix} \theta_0 = -5.0019 \\ \theta_1 = 0.0966 \end{matrix}$$

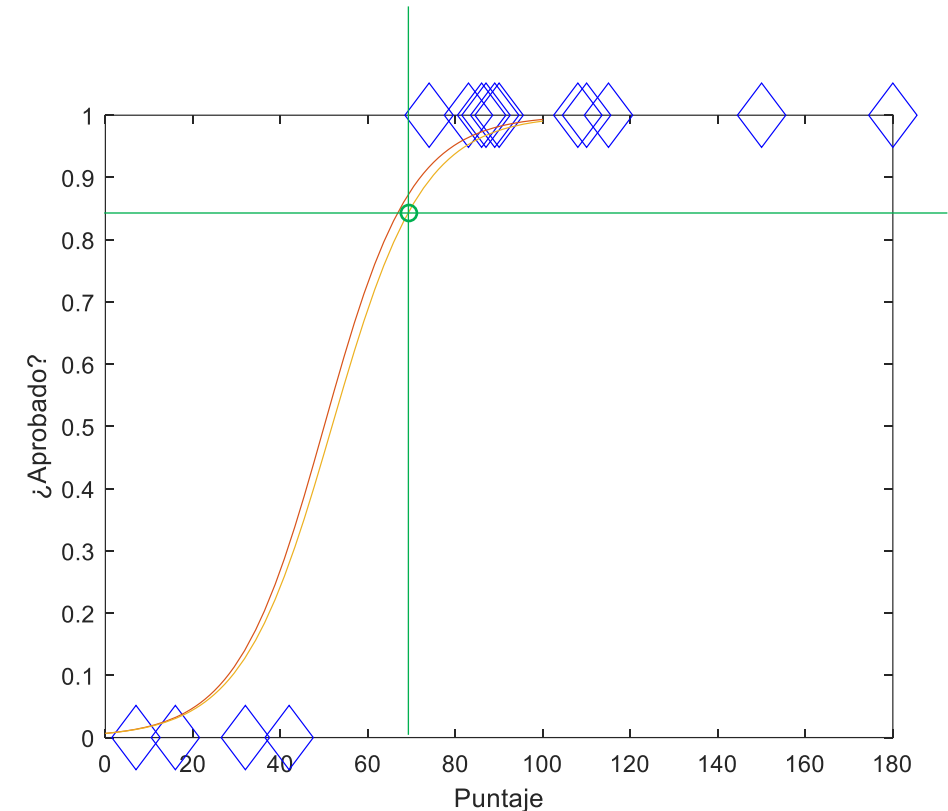
$$h_{\theta}(x) = \frac{1}{1 + e^{-(-5.0019+0.0966x)}}$$

$$h_{\theta}(70) = \frac{1}{1 + e^{-(-5.0019+0.0966(70))}} = \mathbf{0.8529}$$

‡ Luego, debemos redondear el valor y obtendremos la clase:

$$clase(h_{\theta}(70)) = round(0.8529) = \mathbf{1}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0+\theta_1x_1+\theta_2x_2+\dots)}}$$



† Cuando un clasificador toma una decisión, pueden suceder varias cosas:

‡ Decisión **correcta**:

La etiqueta asignada es **1** y debía ser 1 (verdadero positivo, **VP**)

La etiqueta asignada es **0** y debía ser 0 (verdadero negativo, **VN**)

‡ Decisión **incorrecta**:

La etiqueta asignada es **1** y debía ser 0 (falso positivo, **FP**)

La etiqueta asignada es **0** y debía ser 1 (falso negativo, **FN**)

† Y entonces, ¿Cómo podemos saber si nuestro modelo se desempeña bien o no?

‡ Podemos definir varias métricas a partir de esos 4 escenarios, y para ello debemos concentrar los resultados en un matriz, llamada matriz de confusión.

† La matriz de confusión se organiza de la siguiente manera:

		Predicción	
		Sí	No
Real	Sí	VP ✓	FN ✗
	No	FP ✗	VN ✓

† De aquí podemos definir distintas métricas:

- ‡ Exactitud (accuracy)
- ‡ Precisión (precision)
- ‡ Exhaustividad (recall)
- ‡ F1-score

- † La exactitud es bastante común, pero es engañosa si los datos están desbalanceados:

$$\left(\frac{\text{aciertos}}{\text{total}} \right) \quad accuracy = \frac{VP + VN}{VP + VN + FP + FN}$$

- † La precisión estima la calidad del modelo al clasificar una muestra como positivo:

$$\left(\frac{\text{positivos correctos}}{\text{total positivos}} \right) \quad precision = \frac{VP}{VP + FP}$$

- † La exhaustividad indica qué tan bueno es para detectar la clase:

$$\left(\frac{\text{positivos correctos}}{\text{positivos reales}} \right) \quad recall = \frac{VP}{VP + FN}$$

- † F1-score combina precisión y exhaustividad en un solo indicador:

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}$$

		Predicción	
		Sí	No
Real	Sí	VP ✓	FN ✗
	No	FP ✗	VN ✓

CLASIFICACIÓN

MATRIZ DE CONFUSIÓN

† **Ejemplo:** Imagine que un clasificador obtiene los siguientes resultados:

Attendance	Homework	Pass	Prediction
80	75	yes	yes
65	70	no	no
95	85	yes	yes
95	100	yes	no
85	65	no	no
75	55	no	no
90	90	yes	yes
65	80	yes	no

$$accuracy = \frac{VP + VN}{VP + VN + FP + FN}$$

$$precision = \frac{VP}{VP + FP}$$

$$recall = \frac{VP}{VP + FN}$$

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}$$

‡ Calcule las métricas de desempeño del clasificador

		Predicción	
		Sí	No
Real	Sí	VP ✓	FN ✗
	No	FP ✗	VN ✓

† Lo primero que debemos hacer es obtener la matriz de confusión:

Attendance	Homework	Pass	Prediction
80	75	yes	yes
65	70	no	no
95	85	yes	yes
95	100	yes	no
85	65	no	no
75	55	no	no
90	90	yes	yes
65	80	yes	no



		Predicción	
		yes	no
Clase real	yes	3	2
	no	0	3

$$accuracy = \frac{VP + VN}{VP + VN + FP + FN} = \frac{6}{8} = 75\%$$

$$precision = \frac{VP}{VP + FP} = \frac{3}{3} = 100\% \quad recall = \frac{VP}{VP + FN} = \frac{3}{5} = 60\%$$

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall} = \frac{2(1)(0.6)}{1.6} = 75\%$$

		Predicción	
		Sí	No
Real	Sí	VP ✓	FN ✗
	No	FP ✗	VN ✓

- † Al igual que una clasificación binaria, la clasificación multiclase nos permite asignar una etiqueta a una muestra (conjunto de características)
- † Sin embargo, a diferencia del caso binario, en el caso multiclase pueden haber **más de dos etiquetas**
- † Por tanto, ya no se trata solamente de saber si una muestra pertenece o no a una clase, sino de identificar a **cuál** de todas las posibles clases pertenece
- † Esto lleva a que la **regresión logística** que hemos venido utilizando se vuelva **inservible**
- † Pero, hay formas de darle la vuelta al problema...
 - ... Por ejemplo, **descomponiendo** el problema **multiclase** en varios problemas de clasificación **binaria**...

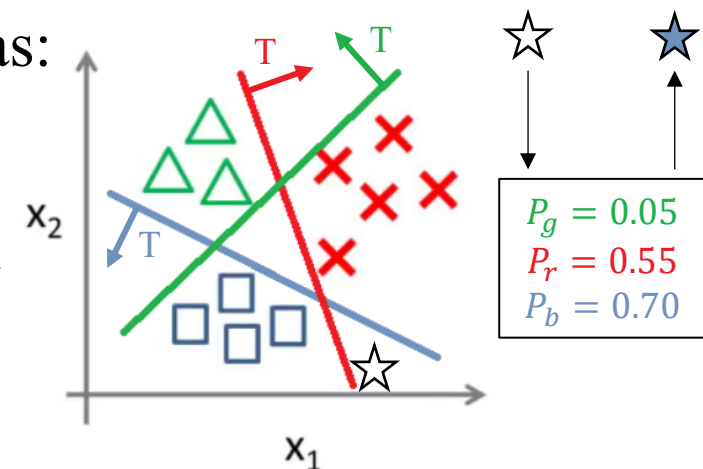
Esto se hace a través de una técnica conocida como one-hot encoding

- † La técnica de *1-hot encoding* tiene varias ventajas:
 - ‡ Nos permite usar regresión logística (clasificador binario) para hacer clasificación multiclase.
 - ‡ Mejora el desempeño de clasificadores multiclase cuando las clases no son secuenciales (e.g. rojo, verde, azul).
- † La idea es simple: **entrenar** un clasificador **binario** que sea capaz de detectar **una clase** en particular, y quedarse con el que ofrezca la mayor probabilidad.
- † Esto requiere descomponer la salida (clase) en varias columnas:

id	color
1	red
2	blue
3	green
4	blue



id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0



- † Sin embargo, al tener múltiples clases, nuestra matriz de confusión crece y se vuelve más ... confusa ...
- ‡ Cada fila contiene las muestras que pertenecen a una clase en particular
- ‡ Cada columna contiene todas las predicciones de una clase dada
- ‡ La diagonal contiene las predicciones correctas de cada clase
- † Por ejemplo, para un problema con tres clases podemos tener algo como:

		Predicción			
		C1	C2	C3	
Clase real	C1	7	1	3	Correcto Incorrecto
	C2	8	2	2	
	C3	9	3	1	

† De esta manera, nuestras definiciones deben aplicarse por clase:

‡ Predicciones **correctas**:

TP: Se estima que pertenece y efectivamente pertenece a la clase

TN: Se estima que no pertenece y efectivamente no pertenece a la clase

‡ Predicciones **incorrectas**:

FP: Se estima que pertenece, pero realmente no pertenece a esa clase

FN: Se estima que no pertenece, pero realmente sí pertenece a la clase

† Es decir, tendremos un conjunto de valores por clase

CLASIFICACIÓN

MATRIZ DE CONFUSIÓN MULTICLASE

† Por ejemplo, para nuestros datos tenemos que...

‡ Clase 1:

$$\mathbf{TP} = 7$$

$$\mathbf{TN} = 2 + 2 + 3 + 1 = 8$$

$$\mathbf{FP} = 8 + 9 = 17$$

$$\mathbf{FN} = 1 + 3 = 4$$

† Por tanto, podemos calcular las métricas de desempeño:

$$\text{Accuracy} = (\mathbf{TP} + \mathbf{TN}) / (\mathbf{TP} + \mathbf{TN} + \mathbf{FP} + \mathbf{FN}) = 15/36 = 0.42$$

$$\text{Precision} = \mathbf{TP} / (\mathbf{TP} + \mathbf{FP}) = 7/24 = 0.29$$

$$\text{Recall} = \mathbf{TP} / (\mathbf{TP} + \mathbf{FN}) = 7/11 = 0.64$$

$$\text{F1-Score} = 2 (0.29) (0.64) / (0.29 + 0.64) = 0.40$$

		Predicción				
		C1	C2	C3		
Clase real	C1	7	1	3		Correcto Incorrecto
	C2	8	2	2		
	C3	9	3	1		

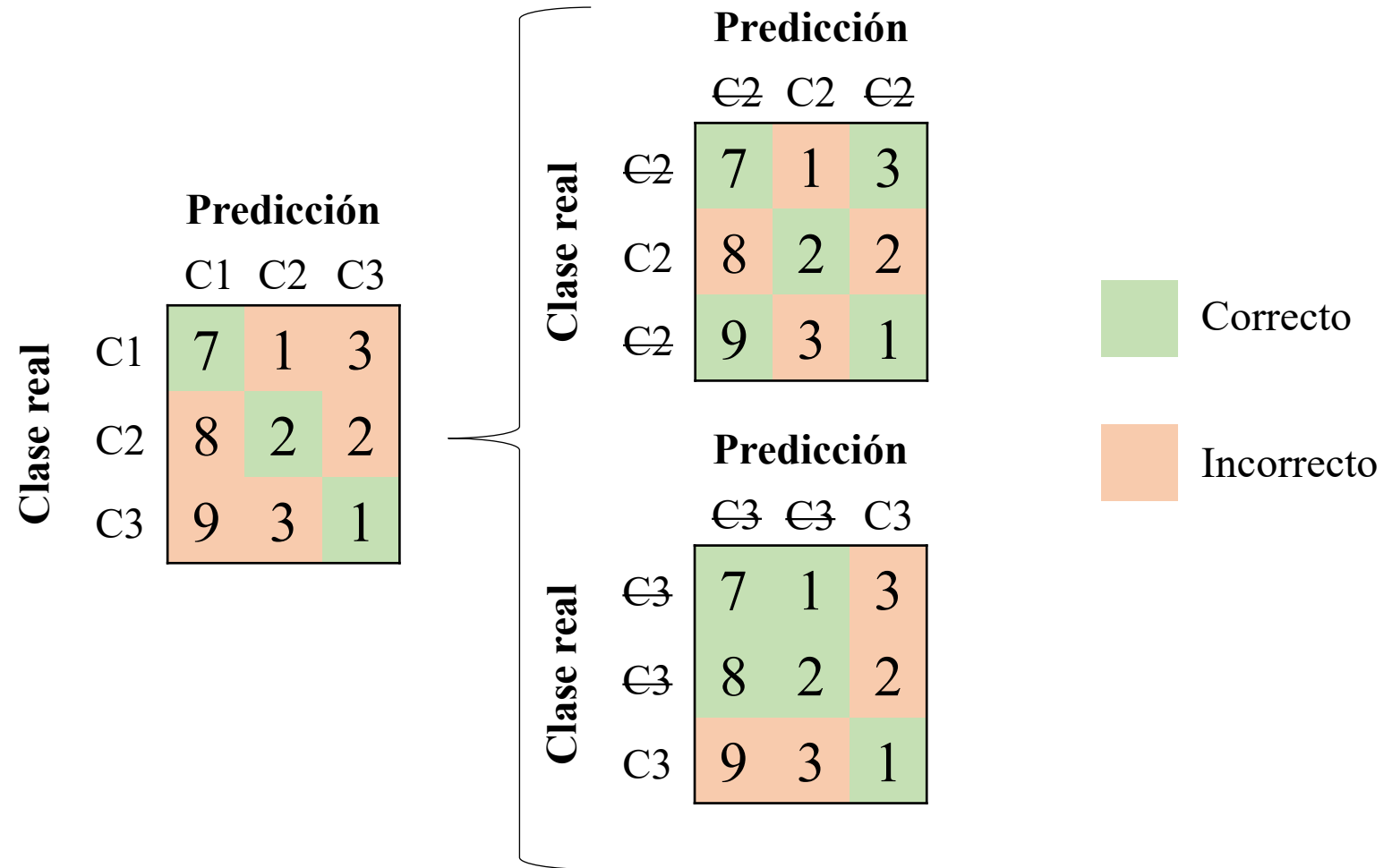


		Predicción				
		C1	€1	€1		
Clase real	C1	7	1	3		Correcto Incorrecto
	€1	8	2	2		
	€1	9	3	1		

CLASIFICACIÓN

MATRIZ DE CONFUSIÓN MULTICLASE

† ¿Cuál es el valor de las métricas para las otras clases?

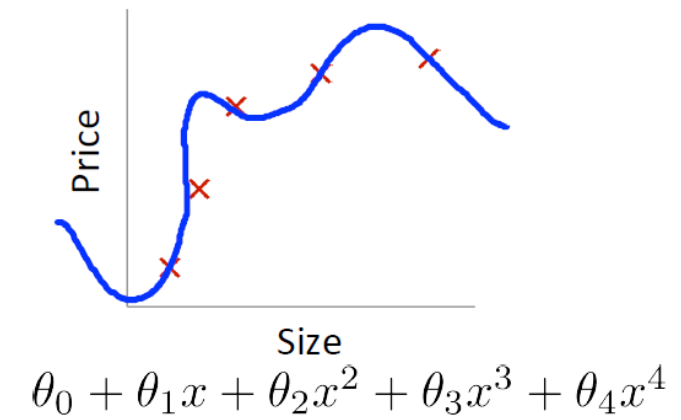
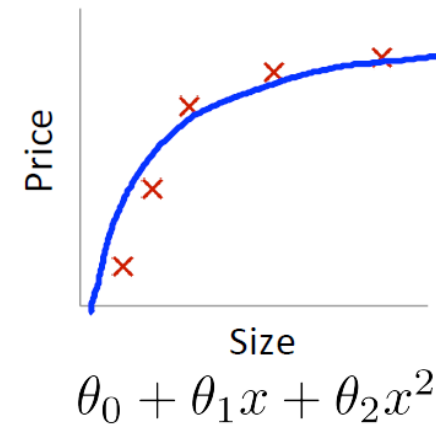
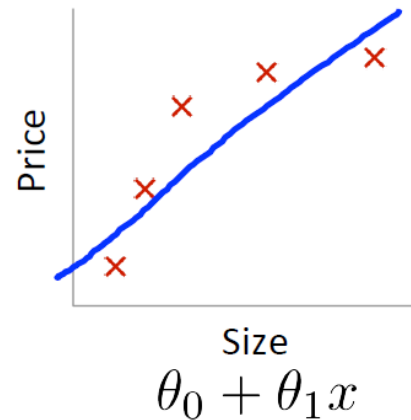


- † De esta manera podemos evaluar la calidad del clasificador al intentar detectar cada una de las clases
- † También podemos dar un puntaje global, relacionado con el F1-Score:
 - ‡ Micro F1-Score: Calculado con la tabla global (no recomendado)
 - ‡ Macro F1-Score: Promedia los valores de F1-Score de cada clasificador
 - ‡ Weighted F1-Score: Pondera usando como pesos el total de muestras de la clase

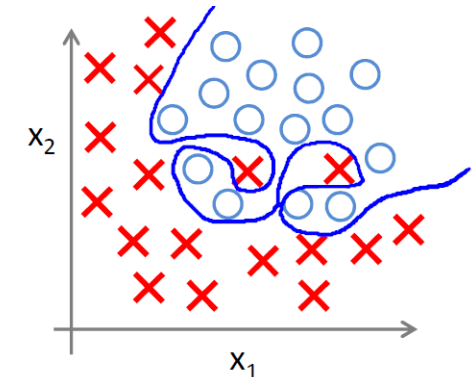
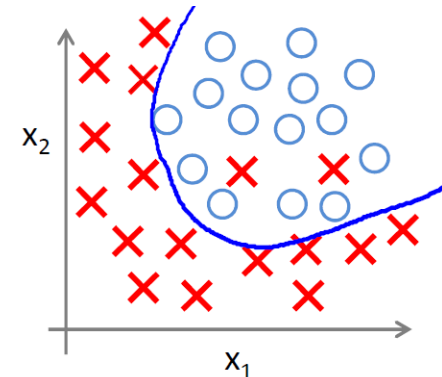
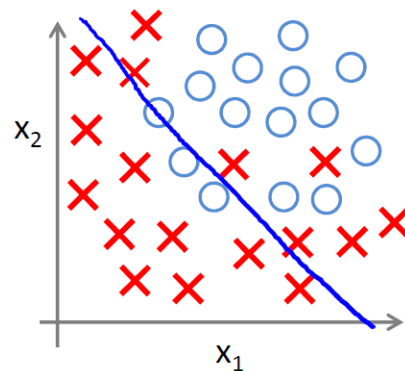
		Predicción				
		C1	C2	C3		
Clase real	C1	7	1	3		Correcto
	C2	8	2	2		
	C3	9	3	1		
					Incorrecto	

† Al igual que con regresión, cuando hacemos clasificación multivariable logramos crear comportamientos no lineales, por lo que podemos ajustarnos mejor a los datos. A título de comparación, tenemos que:

Regresión



Clasificación



- † Dado que la única diferencia entre regresión lineal y regresión logística es la función de hipótesis, en este caso también tendremos que:

$$h_{\theta} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

- † Y de esta manera, GD se mantiene exactamente igual al caso de regresión lineal:

$$\theta_0 = \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_i) - y_i) \quad \theta_k = \theta_k - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x_{i,k}) - y_i) x_{i,k} \quad \forall k \in \{1, 2, 3, 4\}$$

- † Solamente debemos tener en cuenta que nuestra función de hipótesis sigue siendo la ecuación de la regresión logística multivariable:

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots)}}$$

`sklearn.linear_model.LogisticRegression`

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0,
warm_start=False, n_jobs=None, l1_ratio=None) ⓘ
```

[\[source\]](#)

† Permite hacer clasificación binaria (one-vs-rest, OvR) o multiclase:

multi_class : {'auto', 'ovr', 'multinomial'}, default='auto'

If the option chosen is 'ovr', then a binary problem is fit for each label. For 'multinomial' the loss minimised is the multinomial loss fit across the entire probability distribution, *even when the data is binary*. 'multinomial' is unavailable when solver='liblinear'. 'auto' selects 'ovr' if the data is binary, or if solver='liblinear', and otherwise selects 'multinomial'.

† También permite agregar penalización (reducir overfitting):

penalty : {'l1', 'l2', 'elasticnet', None}, default='l2'

Specify the norm of the penalty:

- `None`: no penalty is added;
- `'l2'`: add a L2 penalty term and it is the default choice;
- `'l1'`: add a L1 penalty term;
- `'elasticnet'`: both L1 and L2 penalty terms are added.

Warning: Some penalties may not work with some solvers. See the parameter `solver` below, to know the compatibility between the penalty and solver.

`sklearn.linear_model.LogisticRegression`

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0,
warm_start=False, n_jobs=None, l1_ratio=None) ¶
```

[\[source\]](#)

† Puede autoajustar pesos para variables en diferentes rangos:

`class_weight` : dict or 'balanced', default=None

Weights associated with classes in the form `{class_label: weight}`. If not given, all classes are supposed to have weight one.

The "balanced" mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

† Así como definir una semilla para los métodos basados en aleatorios:

`random_state` : int, RandomState instance, default=None

Used when `solver` == 'sag', 'saga' or 'liblinear' to shuffle the data. See [Glossary](#) for details.

sklearn.linear_model.LogisticRegression

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0,
warm_start=False, n_jobs=None, l1_ratio=None) †
```

[\[source\]](#)

† Incluye diferentes solvers para optimizar el modelo:

solver : {'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs'

Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- For multiclass problems, only 'newton-cg', 'sag', 'saga' and 'lbfgs' handle multinomial loss;
- 'liblinear' is limited to one-versus-rest schemes.
- 'newton-cholesky' is a good choice for `n_samples >> n_features`, especially with one-hot encoded categorical features with rare categories. Note that it is limited to binary classification and the one-versus-rest reduction for multiclass classification. Be aware that the memory usage of this solver has a quadratic dependency on `n_features` because it explicitly computes the Hessian matrix.

Warning: The choice of the algorithm depends on the penalty chosen. Supported penalties by solver:

- 'lbfgs' - ['l2', None]
- 'liblinear' - ['l1', 'l2']
- 'newton-cg' - ['l2', None]
- 'newton-cholesky' - ['l2', None]
- 'sag' - ['l2', None]
- 'saga' - ['elasticnet', 'l1', 'l2', None]

¿Preguntas?

- † Para los datos mostrados, implemente un clasificador (usando Scikit-learn) que estime si un estudiante aprueba o no el curso:
 - ‡ Considerando solamente la columna ‘Attendance’
 - ‡ Considerando solamente la columna ‘Homework’
 - ‡ Considerando ambas columnas
- † Calcule las métricas de desempeño y compare los clasificadores. ¿Cuál es mejor?

Attendance	Homework	Pass
80	75	yes
65	70	no
95	85	yes
95	100	yes
85	65	no
75	55	no
90	90	yes
65	80	yes

$$accuracy = \frac{VP + VN}{VP + VN + FP + FN}$$

$$precision = \frac{VP}{VP + FP}$$

$$recall = \frac{VP}{VP + FN}$$

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall}$$

		Predicción	
		A	B
Real	A	VP ✓	FN ✗
	B	FP ✗	VN ✓